Milestone 1 Report — From Pixels to Language

CSE485 / CSE488 Deep Learning · Nov 3, 2026 · @Moaz Darwish

1. Introduction

Our best model, an ImageNet-pretrained ResNet-50 fine-tuned on CUB-200-2011, reaches 81.7 % validation accuracy on 200 bird species, against 46.7 % for a CNN trained from scratch.

The project trains one CNN encoder on bird images and reuses it for three tasks: species classification, image colorization and image captioning. Milestone 1 covers the first part: understanding the data and training the encoder as a classifier, both from scratch and with transfer learning.

Dataset. We use CUB-200-2011 with its captions, from Hugging Face (CUB_train, CUB_test): 200 species, 5,994 training and 5,794 test images, and 10 captions per image. It is a fine-grained problem: many species differ only in small details such as wing bars, eye rings or bill shape, and there are only about 30 training images per species. That makes it a good test of how much pretraining helps, and the captions and colours make it suitable for the later colorization and captioning tasks.

Code. Everything runs from one command-line interface (`python -m src.cli`) driven by YAML configs. Training ran on Google Colab T4 GPUs; runs are logged with MLflow. Code: github.com/Moazdarwishh/DeepLearning-Project.

2. Data understanding and EDA

The data is clean and balanced in training, but the test set is uneven, a few images are grayscale, and two validation images are duplicates of test images. We scanned all 11,788 images and 117,880 captions (`python -m src.cli data --action eda`); the figures are in `outputs/eda/figures/` .

Validation split. The dataset has no validation set, so we hold out a stratified 15 % of the training set (seed 42): 5,094 training and 900 validation images, 4–5 per species. The split is saved in `data/splits/` so every team member uses the same images. The test set is only used for the final evaluation.

Finding	Result	Consequence
Corrupt files	0	nothing removed
Images per species	train 25–26, validation 4–5, test 11–30 (Least Auklet 11)	report macro F1 and per-species accuracy, not only accuracy
Image size	mostly 500 px wide, 120–500 px high, aspect ratio 0.4–3.6	resize to 256, crop 224; random crops in training
Grayscale	8 stored as grayscale, 23 more with almost no colour	converted to RGB; to be excluded from colorization later
Image quality	about 1 % each of blurry, dark, bright or low-contrast images	kept; brightness and contrast augmentation
Duplicates	8 confirmed pairs (perceptual hash distance ⩽ 2)	2 validation images that also appear in the test set removed
Label conflict	test images 565 and 2974: the same photo labelled Chuck-will's-widow and Whip-poor-will	kept, to stay comparable with published results
Colour statistics (train)	mean 0.485 / 0.499 / 0.433, std 0.232 / 0.228 / 0.266	normalisation for the scratch CNN

Captions. Every image has exactly 10 captions. They average 15 words; 95 % are 25 words or shorter and the longest has 66. The training vocabulary has 4,127 words, 1,916 of which appear only once. We found 848 broken characters (`�` , broken apostrophes), 639 repeated captions within an image and many typos ("thraot", "yelllow", "blackbeak").

Figures: class distribution (`01`), image sizes (`02`), image quality (`03`), grayscale images (`04`), low-quality examples (`05`), duplicates (`06` , `06b`), sample species and the eight gull species that look alike (`07a` , `07b`), caption lengths (`08`), top words (`09`) and colour words (`10`).

3. Cleaning, preprocessing and augmentation

We keep almost every image, remove only the two test leaks, clean the captions conservatively, and use augmentation that changes position and lighting but never colour.

Images. Grayscale images are converted to RGB. The two validation images that are duplicates of test images 661 and 5115 are dropped (`data/splits/exclusions.json`), leaving 898 validation images; keeping them would let test data influence model selection.

Duplicates inside the training or test set, and the test label conflict, are kept so our test set stays identical to the official one.

Captions (used in Milestone 2, cleaned now). Each `description` is split into its 10 captions, lowercased, and stripped of punctuation and digits. Apostrophes and the broken `` character are deleted, so "bird's" becomes "birds" instead of "bird s". Typos are fixed only for rare words that a spell checker does not know and whose correction is a frequent word in our captions; this protects bird terms such as "superciliary" and "malar". Glued words are split only when the two-word phrase exists in the corpus: "agray" becomes "a gray", but "aalso" becomes "also". The fixes are learned from training captions only. Result: 408 fixes (316 typos, 92 split words) and a vocabulary of 3,719 words instead of 4,127.

Augmentation (training only).

Step	Setting	Why
Random rotation	±10°, filled with the mean colour	photos are taken at slight angles; done before cropping so the crop removes the empty corners
Random resized crop	224 px, keeps 50–100 % of the image	birds vary in size and position; the minimum is 50 %, not torchvision's 8 %, so the bird is rarely cut out
Horizontal flip	p = 0.5	a bird facing left is the same species facing right
Colour jitter	brightness 0.3, contrast 0.3, saturation 0.2, **hue 0**	the EDA found dark and low-contrast photos

We deliberately do not change hue or turn images gray: plumage colour separates many species (for example red vs. orange tanagers) and is the target of colorization in Milestone 2. Vertical flips are not used because birds are not photographed upside down.

Evaluation transform. Resize to 256, centre-crop 224. Pretrained models use ImageNet normalisation; the scratch CNN uses our own training-set statistics. Figure `11_augmentations.png` shows four birds with their evaluation view and six augmented versions each.

4. Models

We compare one CNN trained from scratch with ImageNet-pretrained ResNet-50 used in two ways: frozen (linear probe) and fully fine-tuned.

Shared design. Every encoder returns the feature maps of its four stages (at 1/4, 1/8, 1/16 and 1/32 of the input size). The classifier averages the last stage over space and applies dropout and a linear layer with 200 outputs. Returning all four stages now lets Milestone 2 reuse the same encoder as a U-Net encoder for colorization and as an image feature grid for captioning. Models are created by name from the config through a small registry, so the training code does not change when the model does.

Scratch CNN (`scratch_cnn` , 11.3 M parameters, Kaiming initialisation):

Part	Layers	Output (224 px input)
Stem	3×3 conv, stride 2, 32 channels; 3×3 conv, 64 channels	112 × 112 × 64
Stage 1	2 residual blocks, 64 channels, first block stride 2	56 × 56 × 64
Stage 2	2 residual blocks, 128 channels	28 × 28 × 128
Stage 3	2 residual blocks, 256 channels	14 × 14 × 256
Stage 4	2 residual blocks, 512 channels	7 × 7 × 512
Head	global average pooling, dropout 0.4, linear 512 → 200	200 scores

Each residual block has two 3×3 convolutions with batch normalisation and ReLU, plus a skip connection (a 1×1 projection when the shape changes). The skip connections can be switched off in the config to get a plain CNN of the same size.

ResNet-50 (torchvision, ImageNet weights, 23.5 M parameters in the backbone). Its 1000-class head is replaced by our 200-class head.

- **Linear probe:** the backbone is frozen and only the head is trained (410 K parameters). The frozen backbone also stays in evaluation mode, so its batch-normalisation statistics are not changed by our small dataset.
- **Fine-tuning:** all 23.9 M parameters are trained. The backbone gets a learning rate 10× smaller than the head, so the pretrained features are adjusted gently while the new head learns quickly.

5. Training setup and experiments

All three models were trained with the same training loop and the same data; only the settings below differ. Pretrained models learn in a few epochs, while the scratch CNN needs about 45 epochs to approach its best score.

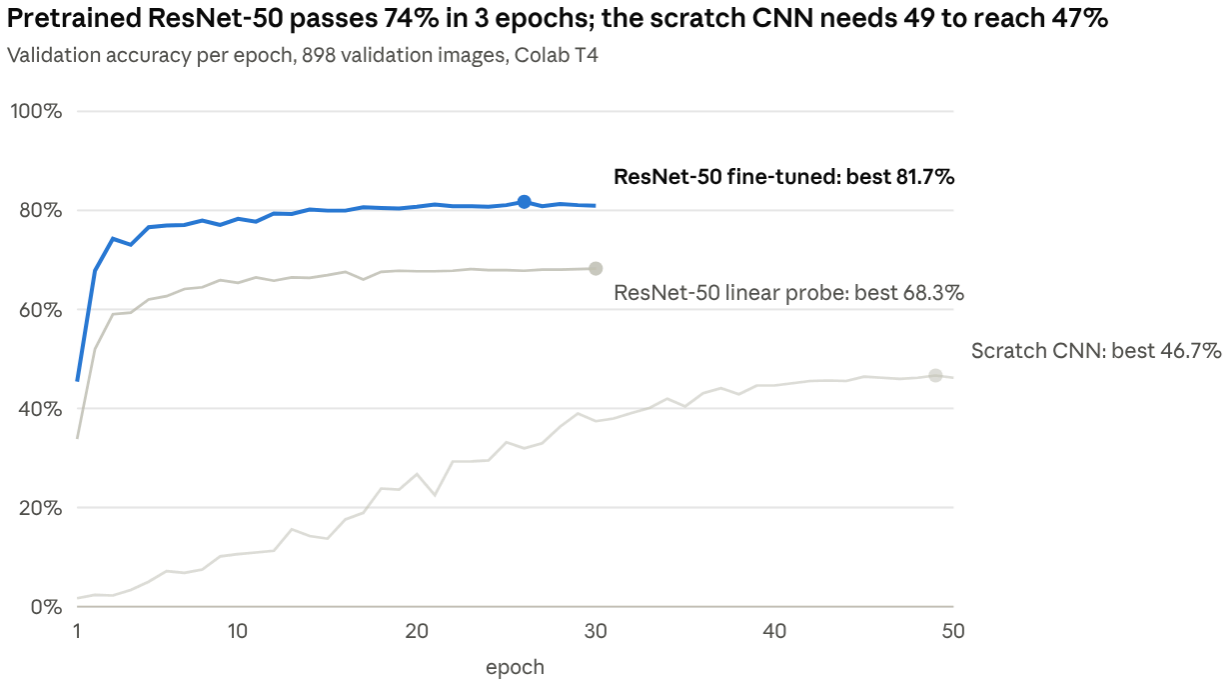
Common to all runs. AdamW, no weight decay on biases and batch-normalisation weights, a linear learning-rate warm-up followed by cosine decay (updated every batch), cross-entropy with label smoothing 0.1, mixed precision on the GPU, seed 42. The best checkpoint is chosen on validation accuracy (validation loss breaks ties), and early stopping ends a run when validation accuracy stops improving. Label smoothing is used because with about 25 images per species a model quickly becomes over-confident.

Run	Config	Batch	Learning rate	Weight decay	Dropout	Warm-up	Epochs	Time per epoch (T4)
Scratch CNN	`cls_scratch.yaml`	64	1e-3	5e-4	0.4	3 epochs	50	67 s
ResNet-50 linear probe	`cls_resnet50_probe.yaml`	64	3e-3 (head)	1e-4	0.3	1 epoch	30	66 s
ResNet-50 fine-tuned	`cls_resnet50.yaml`	32	1e-3 head, 1e-4 backbone	1e-4	0.3	1 epoch	30	79 s

No run was stopped early; each run's best epoch was near its end (49 of 50, 30 of 30, 26 of 30). All three runs take about the same time per epoch even though the probe trains 28× fewer parameters: the bottleneck is Colab's two CPU cores decoding and augmenting JPEG images, not the GPU.

Learning curves. Each run writes `curves.png` (loss, accuracy, top-5 / macro F1 and learning rate per epoch). The chart below compares validation accuracy across the three runs.

- The scratch CNN reaches 10 % after 9 epochs, 30 % after 25 and 46.7 % at epoch 49. Training accuracy ends at 81 %, so it overfits even with augmentation.
- The linear probe is at 52 % after 2 epochs and levels off at 68 %. Training accuracy reaches 94 %.
- The fine-tuned ResNet-50 is at 74 % after 3 epochs and improves slowly to 81.7 % at epoch 26, while its training accuracy is close to 100 % from epoch 12. It memorises the training images, yet validation accuracy keeps rising: the pretrained features, augmentation and label smoothing keep it generalising.



Colab training logs · validation accuracy per epoch, 3 runs

Both pretrained models are ahead of the scratch CNN's final score after the first two epochs.

6. Hyperparameter tuning

We compared Grid Search and Random Search with exactly the same budget, model and search ranges, so the only difference is how the settings are chosen.

Setup. Both searches tune the ResNet-50 linear probe, which is cheap to train because only the head learns. Each trial trains for 5 epochs and is scored on validation accuracy; the test set is never used. Results are saved after every trial, so a disconnected Colab session continues where it stopped. Commands: `python -m src.cli tune --config configs/tune_grid.yaml` (and `tune_random.yaml`), then `tune --compare`.

	Grid Search	Random Search
Learning rate	1e-3, 1e-2	8 values, log-uniform in [1e-3, 1e-2]
Weight decay	1e-4, 1e-2	8 values, log-uniform in [1e-4, 1e-2]
Dropout	0.2, 0.5	8 values, uniform in [0.2, 0.5]
Budget	2 × 2 × 2 = 8 trials × 5 epochs	8 trials × 5 epochs

With the same 8 trials, Grid Search tests only 2 values of each hyperparameter, while Random Search tests 8. When some hyperparameters matter much more than others (often the learning rate), Random Search explores the important ones more densely (Bergstra and Bengio, 2012).

Grid Search results. All 8 trials finished in 45.5 minutes on a Colab T4 (about 66 s per epoch). Validation accuracy after 5 epochs:

Trial	Learning rate	Weight decay	Dropout	Val accuracy	Val top-5	Val macro F1
1	0.001	0.0001	0.2	55.9%	85.8%	0.527
2	0.001	0.0001	0.5	54.0%	84.2%	0.509
3	0.001	0.01	0.2	55.9%	85.8%	0.527
4	0.001	0.01	0.5	54.0%	84.2%	0.509
5	**0.01**	**0.0001**	**0.2**	**65.1%**	**89.2%**	**0.639**
6	0.01	0.0001	0.5	63.3%	87.4%	0.620
7	0.01	0.01	0.2	65.1%	89.2%	0.639
8	0.01	0.01	0.5	63.3%	87.4%	0.619

The best setting is learning rate 0.01, weight decay 0.0001 and dropout 0.2 (trial 5, 65.1%), found after 28.7 of the 45.5 minutes.

The grid shows clearly how much each hyperparameter matters:

- **Learning rate** is by far the most important: going from 0.001 to 0.01 adds about 9 points (55.9% → 65.1%). With only 5 epochs, the smaller learning rate has not converged yet; its training accuracy is still 72% against 89%.
- **Dropout** matters a little: 0.2 is about 2 points better than 0.5 for both learning rates. In such a short run, stronger dropout mostly slows learning down.
- **Weight decay** has almost no effect: each pair of trials that differs only in weight decay ends within 0.1 points (trials 1 and 3, 5 and 7). AdamW shrinks the weights by a factor of $(1 - \mathrm{lr} \cdot \mathrm{wd})$ per step. With 395 steps, wd = 0.01 and the warm-up and cosine schedule, the weights shrink by less than 4% in total, which is too little to change the result.

This means half of the grid budget (4 of 8 trials) went into repeating results that were already known. This is exactly the weakness of Grid Search that Bergstra and Bengio describe. The 65.1% after 5 epochs is lower than the 68.3% of the full probe run because that run trained for 30 epochs. The search is meant to compare settings, not to produce a final model.

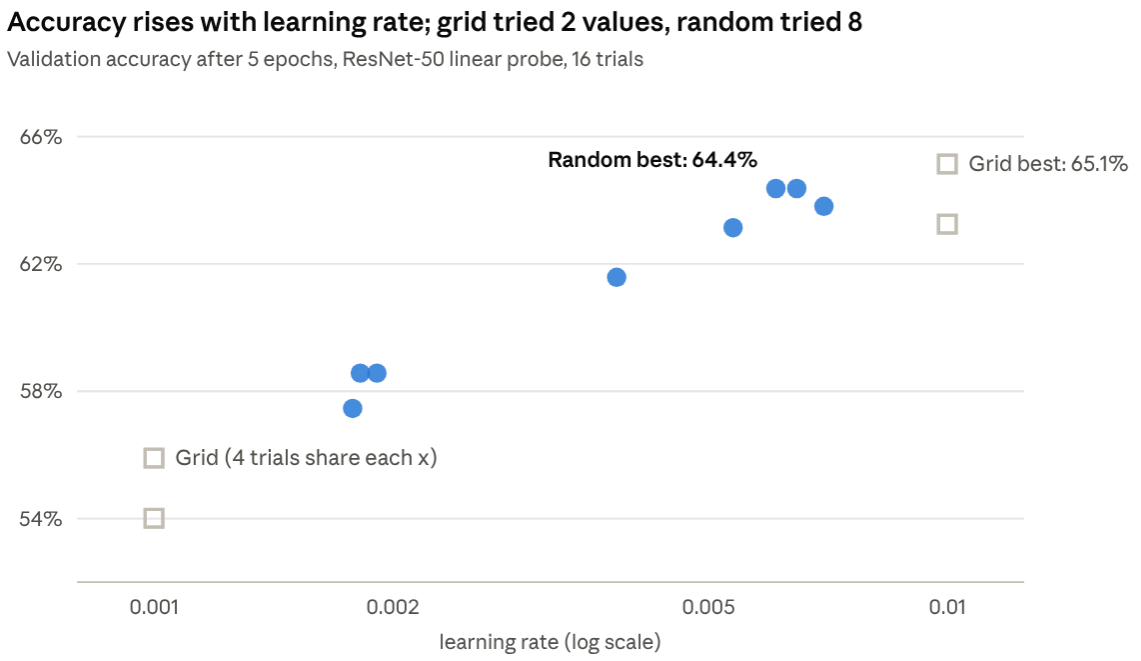
Random Search results. The 8 trials finished in 45.2 minutes. Learning rate and weight decay were drawn on a log scale and dropout uniformly, with seed 0:

Trial	Learning rate	Weight decay	Dropout	Val accuracy	Val top-5	Val macro F1
1	0.00699	0.00328	0.326	63.8%	88.3%	0.627
2	0.00182	0.00105	0.321	58.6%	86.2%	0.556
3	**0.00608**	**0.000404**	**0.343**	**64.4%**	**87.9%**	**0.630**
4	0.00383	0.00655	0.351	61.6%	87.9%	0.598
5	0.00191	0.00325	0.386	58.6%	86.2%	0.559
6	0.00178	0.0066	0.495	57.5%	86.3%	0.548
7	0.00646	0.00637	0.293	64.4%	88.3%	0.631
8	0.00537	0.00628	0.405	63.1%	87.0%	0.617

Trial 8 is the only one whose best epoch was 4, not 5. The best setting is learning rate 0.00608, weight decay 0.000404 and dropout 0.343 (trial 3, 64.4%; trial 7 ties).

Comparison. Both searches used the same 8 trials × 5 epochs and almost the same time:

	Grid Search	Random Search
Best val accuracy	**65.1%**	64.4%
Mean val accuracy	59.6%	**61.5%**
Worst val accuracy	54.0%	**57.5%**
Trials to reach best	5 of 8	**3 of 8**
Minutes to reach best	28.7	**17.1**
Total minutes	45.5	45.2
Distinct values tried (lr / wd / dropout)	2 / 2 / 2	**8 / 8 / 8**
Best setting	lr 0.01, wd 0.0001, dropout 0.2	lr 0.00608, wd 0.000404, dropout 0.343



outputs/tuning/tune-grid and tune-random · results.json, 16 trials

Grid Search found the slightly better setting, by 0.7 points (about 7 of the 898 validation images). This is within noise: the standard error of an accuracy near 65% on 898 images is about 1.6 points. Random Search was better on the other measures. Its average trial was 2 points better, its worst trial was 3.5 points better, and it reached its best result in 17 minutes instead of 29.

The reason Grid won on the best score is visible in the chart. Accuracy keeps rising with the learning rate across the whole range, so the best region is at the top end, 0.01. The grid places half of its trials exactly on that edge, while the random draws happened to stay below 0.007. Random Search was still more informative: its 8 different learning rates show a clear trend, while the grid's 2 values cannot show one. Because the best results sit at the edge of the range, the range itself was too narrow, and a next search should go above 0.01.

For this model, the conclusion is that the learning rate is the only hyperparameter that matters much in a short linear-probe run. Weight decay has almost no effect and dropout has a small one. Random Search spends its budget better when that is the case, but with only 8 trials, which method wins on the best single score is partly luck.

7. Experiment tracking

Every training run and every tuning trial is logged to MLflow, so any run can be reopened and compared with the others.

For each run MLflow stores the full configuration (every hyperparameter), the device, the git commit, the number of parameters, the dataset sizes, per-epoch metrics (training loss and accuracy, validation loss, accuracy, top-5 accuracy and macro F1, learning rate, epoch time), the final best validation metrics, and the learning-curve figure, history and summary files. Checkpoints (`best.pt`, `last.pt`) are stored in each run folder on Google Drive and referenced by path. Classification runs are in the experiment `pixels-to-language`, tuning trials in `pixels-to-language-tuning`.

MLflow 3 needs a database backend, so runs are stored in `mlflow.db` (SQLite). To browse them: `mlflow ui --backend-store-uri sqlite:///mlflow.db`.

MLflow screenshots. These were taken from the MLflow UI (`mlflow ui --backend-store-uri sqlite:///mlflow.db`) after copying the Colab database to a laptop.

Figure: the three training runs in the `pixels-to-language` experiment, with their durations (40.8, 33.3 and 56.7 minutes).

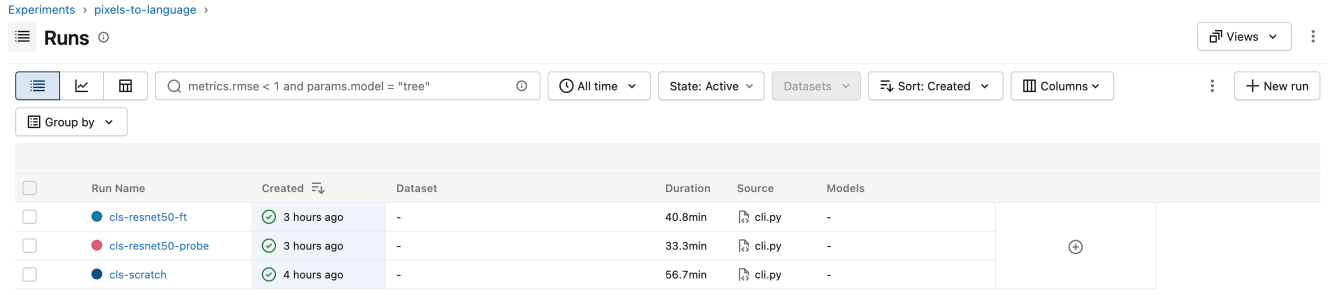


Figure: per-epoch curves logged for the final model, `cls-resnet50-ft`. Training accuracy reaches 1.0 by epoch 10, while validation accuracy and macro F1 keep rising slowly to about 0.81.

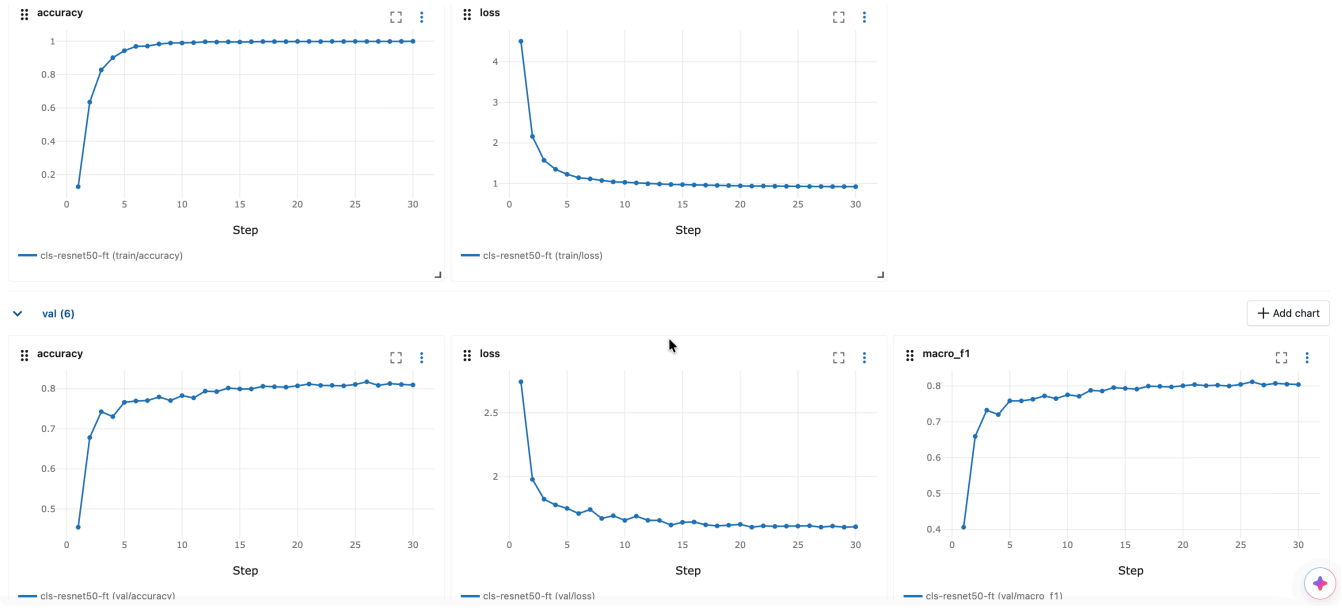


Figure: the MLflow run comparison, best validation metrics. Columns from left to right: fine-tuned, linear probe, scratch.

Metrics

Show diff only

best_val/accuracy	0.817	0.683	0.467
best_val/loss	1.614	2.073	2.801
best_val/macro_f1	0.811	0.671	0.458
best_val/macro_precision	0.841	0.704	0.495
best_val/macro_recall	0.814	0.678	0.463
best_val/top5_accuracy	0.945	0.913	0.748
epoch_time_s	74.68	65.46	76.72

Figure: the same comparison, model parameters. MLflow highlights the rows that differ: frozen or trainable backbone, pretrained or not, model name and number of trainable parameters. It also records the run folder and the git commit (`a3e2ba5`) of the code that produced each run.

model.dropout	0.3	0.3	0.4
model.freeze_backbone	False	True	False
model.name	resnet50	resnet50	scratch_cnn
model.num_classes	200	200	200
model.params	{}	{}	{'widths': [64, 128, 256, 512], 'blocks_per_stage': 2, 'residual': True}
model.params_total	23917832	23917832	11293288
model.params_trainable	23917832	409800	11293288
model.pretrained	True	True	False
run.device	cuda	cuda	cuda
run.dir	outputs/runs/cls-resnet50-ft	outputs/runs/cls-resnet50-probe	outputs/runs/cls-scratch
run.git_commit	a3e2ba5	a3e2ba5	a3e2ba5

Figure: the 16 tuning trials (8 grid, 8 random) in the separate `pixels-to-language-tuning` experiment, about 5.6 minutes each.

Experiments › pixels-to-language-tuning ›

Runs

Views

metrics.rmse < 1 and params.model = "tree" | All time | State: Active | Datasets | Sort: Created | Columns | New run | Group by

Run Name	Created	Dataset	Duration	Source	Models
trial-07	1 hour ago	-	5.7min	cli.py	-
trial-06	1 hour ago	-	5.7min	cli.py	-
trial-05	1 hour ago	-	5.6min	cli.py	-
trial-04	1 hour ago	-	5.6min	cli.py	-
trial-03	1 hour ago	-	5.6min	cli.py	-
trial-02	1 hour ago	-	5.6min	cli.py	-
trial-01	1 hour ago	-	5.7min	cli.py	-
trial-00	1 hour ago	-	5.6min	cli.py	-
trial-07	1 hour ago	-	5.6min	cli.py	-
trial-06	2 hours ago	-	5.6min	cli.py	-
trial-05	2 hours ago	-	5.6min	cli.py	-
trial-04	2 hours ago	-	5.6min	cli.py	-
trial-03	2 hours ago	-	6.0min	cli.py	-
trial-02	2 hours ago	-	5.7min	cli.py	-
trial-01	2 hours ago	-	5.7min	cli.py	-
trial-00	2 hours ago	-	5.6min	cli.py	-

Show more columns (93 total)

16 matching runs

8. Evaluation

The fine-tuned ResNet-50 is best on every metric, on both validation and test: 83.1% test accuracy, 95.6% top-5 and 0.830 macro F1. Its remaining errors are almost all between very similar species of the same group.

Metrics.

- **Accuracy:** the share of images whose top prediction is the correct species. Easy to read, but it gives more weight to species with more test images (11 to 30 per species).
- **Top-5 accuracy:** the share of images whose correct species is among the five highest scores. It shows whether a wrong prediction was "close", which matters for look-alike species such as the eight gulls.
- **Macro precision:** for each species, how many of the images predicted as that species really are it, averaged over species with equal weight. Low precision means many false alarms.
- **Macro recall:** for each species, how many of its images were found, averaged with equal weight. It equals the mean per-species accuracy.
- **Macro F1:** the harmonic mean of precision and recall for each species, averaged over species. Every species counts equally, so the small test classes are not drowned out; it is our main summary number next to accuracy.
- **Confusion matrix and confused pairs:** which species are mistaken for which. A 200 × 200 matrix is hard to read, so we also list the most frequent mistakes with example images.

Validation results (898 images, best epoch of each run):

Model	Trainable parameters	Accuracy	Top-5	Macro precision	Macro recall	Macro F1
Scratch CNN	11.3 M	46.7 %	74.8 %	0.495	0.463	0.458
ResNet-50 linear probe	0.41 M	68.3 %	91.3 %	0.704	0.678	0.671
ResNet-50 fine-tuned	23.9 M	**81.7 %**	**94.5 %**	**0.841**	**0.814**	**0.811**

What the scores mean: the scratch CNN gets about half of the birds right and has the correct species in its top five three times out of four. Pretrained features alone (the probe) add 21.6 points with 28× fewer trained parameters, and adapting the whole network adds another 13.5. In all models macro F1 is close to accuracy, so performance is spread fairly evenly over species rather than coming from a few easy ones.

Test results (5,794 official test images, each model's best-validation checkpoint, evaluated once):

Model	Accuracy	Top-5	Macro precision	Macro recall	Macro F1
Scratch CNN	50.4%	78.2%	0.507	0.505	0.500
ResNet-50 linear probe	69.8%	91.7%	0.706	0.701	0.699
ResNet-50 fine-tuned	**83.1%**	**95.6%**	**0.835**	**0.833**	**0.830**

The ranking is the same as on validation, and all three models score slightly higher on test than on validation (+3.7, +1.5 and +1.4 points). This is expected noise from the small validation set: with 898 images, an accuracy has a standard error of about 1.6 points, against about 0.5 points for the test set. Choosing epochs and settings on validation did not overfit to it, otherwise the test scores would be lower. The comparison figure is saved as `outputs/comparison_test.png`.

Per-species results. Mean per-species accuracy is 50.5% for the scratch CNN, 70.1% for the probe and 83.3% for the fine-tuned model. The spread between species also shrinks (standard deviation 0.21 → 0.19 → 0.16). The scratch CNN never gets one species right (Long-tailed Jaeger, 0%). The fine-tuned model gets every species right at least sometimes and classifies 29 species perfectly.

Most confused pairs, fine-tuned model:

True species	Predicted as	Times	Other direction
Common Tern	Arctic Tern	11	5
Long-tailed Jaeger	Pomarine Jaeger	10	5
Acadian Flycatcher	Least Flycatcher	9	0
American Crow	Fish Crow	8	7
California Gull	Western Gull	8	2

Its hardest species are Common Tern (20%), California Gull (27%), Acadian Flycatcher (41%), American Crow (43%) and Western Wood Pewee (43%).

What the errors show. Every top confusion of the fine-tuned model is between two very similar species in the same group: terns, jaegers, *Empidonax* flycatchers, crows and gulls. Birdwatchers separate many of these by voice, size or seasonal details rather than appearance; American and Fish Crow are known mainly by their calls. Many confusions also go both ways (crows 8 and 7), which means the model cannot tell the pair apart, not that it is biased towards one. The other two models make different kinds of mistakes:

- **Scratch CNN:** it also mixes up pairs from different families, such as Frigatebird → Pomarine Jaeger (9 times) and Nighthawk → Whip-poor-will (8). These are dark birds against the sky and brown night birds, so it relies on overall shape, colour and background rather than identifying details.
- **Linear probe:** its list is dominated by hummingbirds (Anna's, Ruby-throated and Rufous, 9–11 times each way). The frozen ImageNet features do not separate these small, similar birds well. After fine-tuning, the hummingbirds disappear from the top confusions, which shows what adapting the encoder adds.

9. Explainability (Grad-CAM)

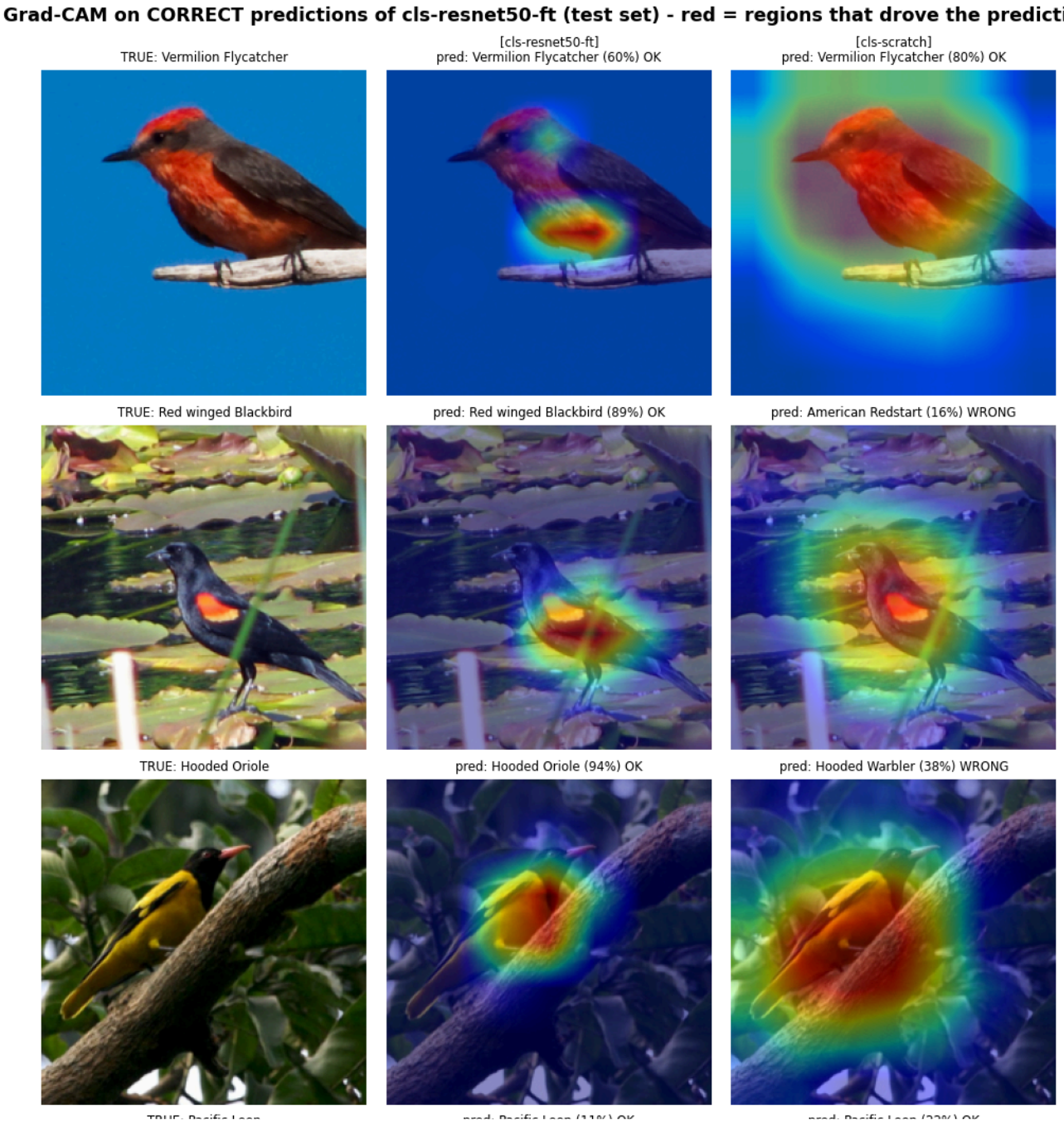
Grad-CAM shows which parts of an image drove a prediction, so we can check whether a model looks at the bird or at the background.

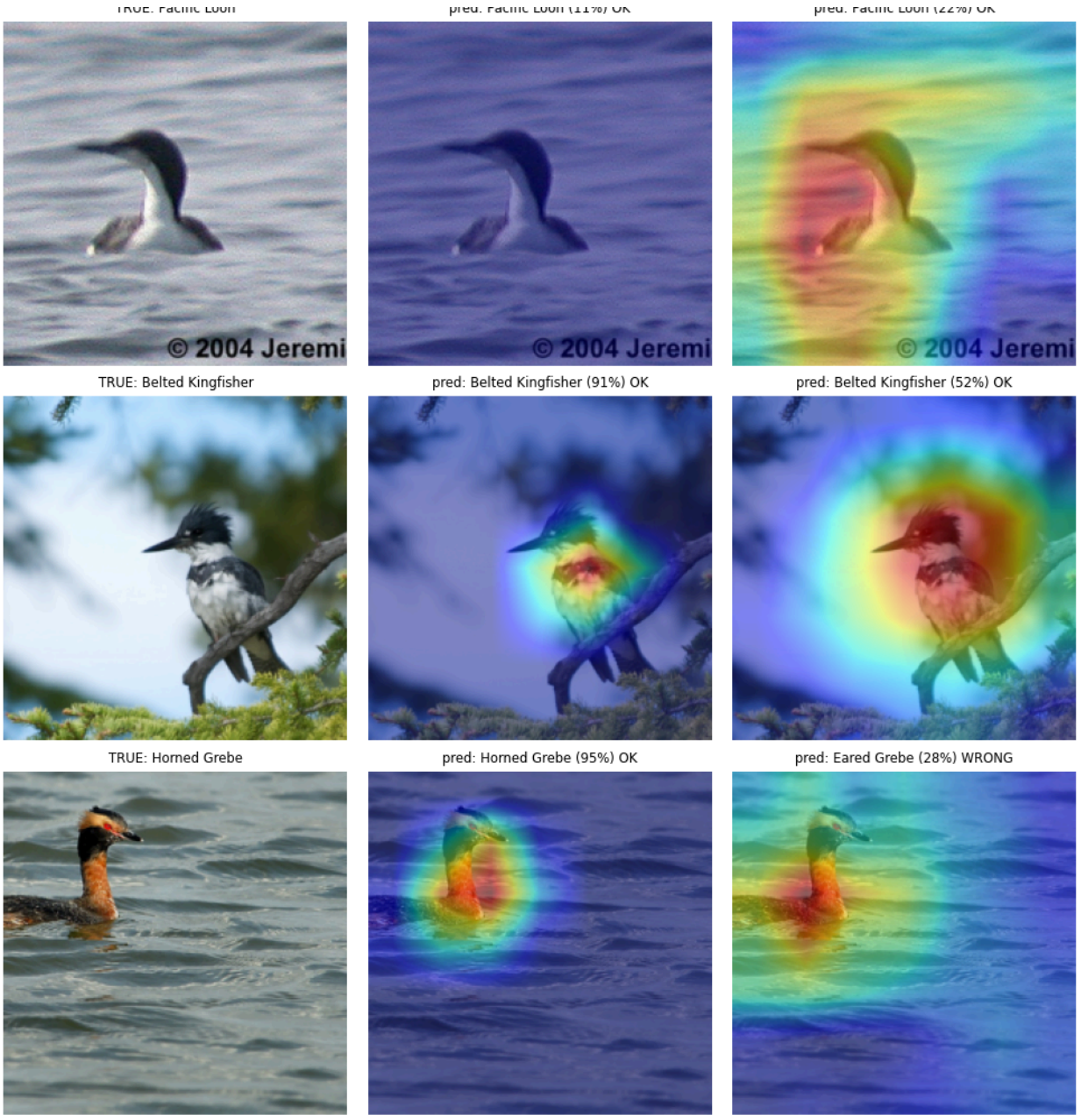
Method. For the predicted class, we take the feature maps of the encoder's last stage and the gradient of the class score with respect to them. Each channel is weighted by its average gradient, the weighted maps are summed, negative values are set to zero, and the result is upsampled to the image size (Selvaraju et al., 2017). The map is only as detailed as the last feature map, which is 7 × 7 for a 224-pixel image, so it shows regions, not pixels. It also works for the frozen probe because gradients are taken with respect to the input. We checked the implementation against the exact formula for a global-average-pooling + linear head.

Setup. `python -m src.cli gradcam --checkpoint <fine-tuned> <scratch> --split test` picks random correct and random wrong predictions of the fine-tuned model and shows the same images under both models, each with its prediction and confidence.

We ran Grad-CAM on 6 random test images that the fine-tuned model classified correctly and 6 that it got wrong (`outputs/gradcam/cls-resnet50-ft_vs_cls-scratch/`). Each row shows the image, the fine-tuned model's map and the scratch CNN's map; red marks the regions that pushed the score of the predicted class up.

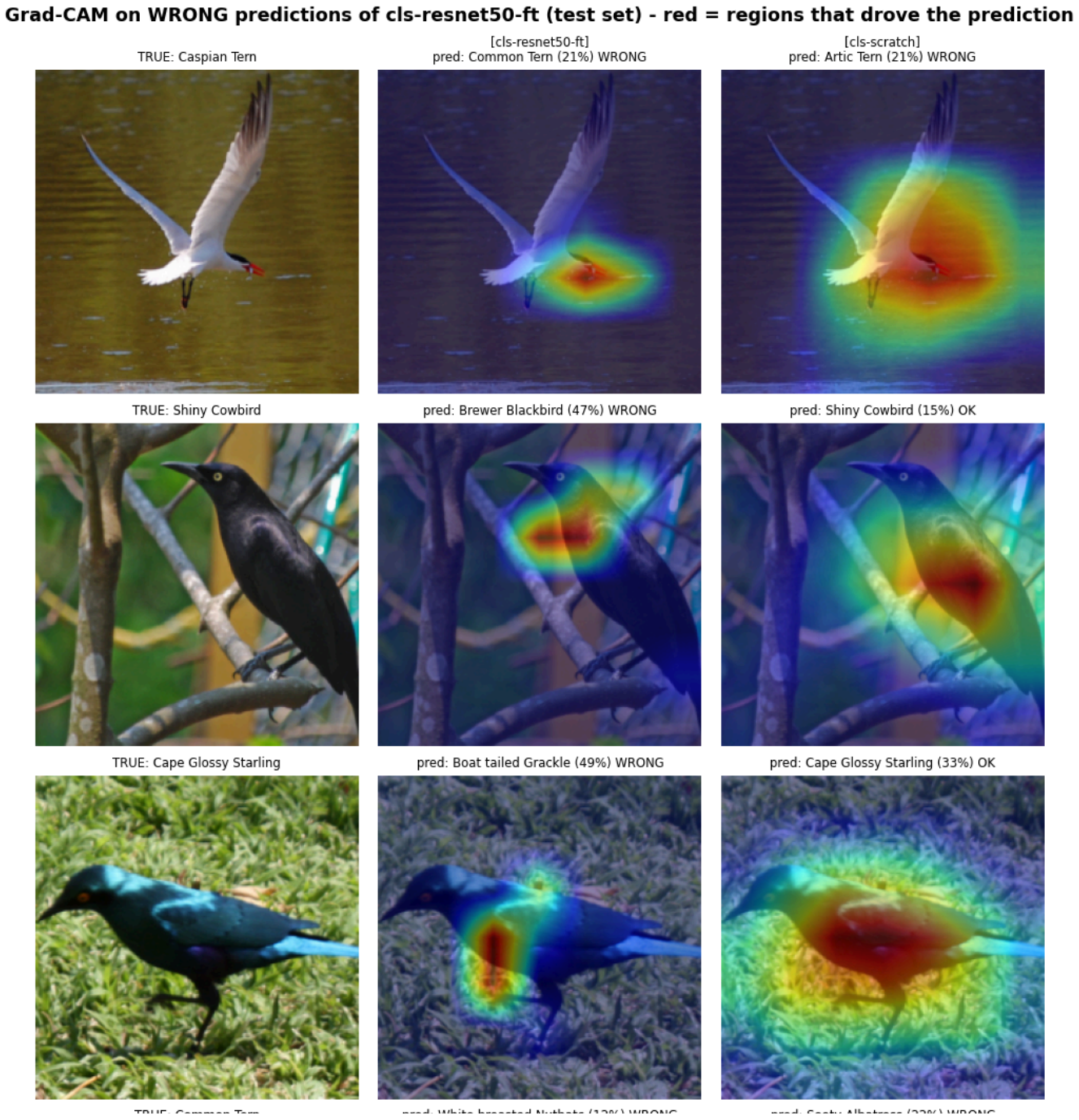
Figure: Grad-CAM on correct predictions of the fine-tuned model
(`gradcam_test_correct.png`)

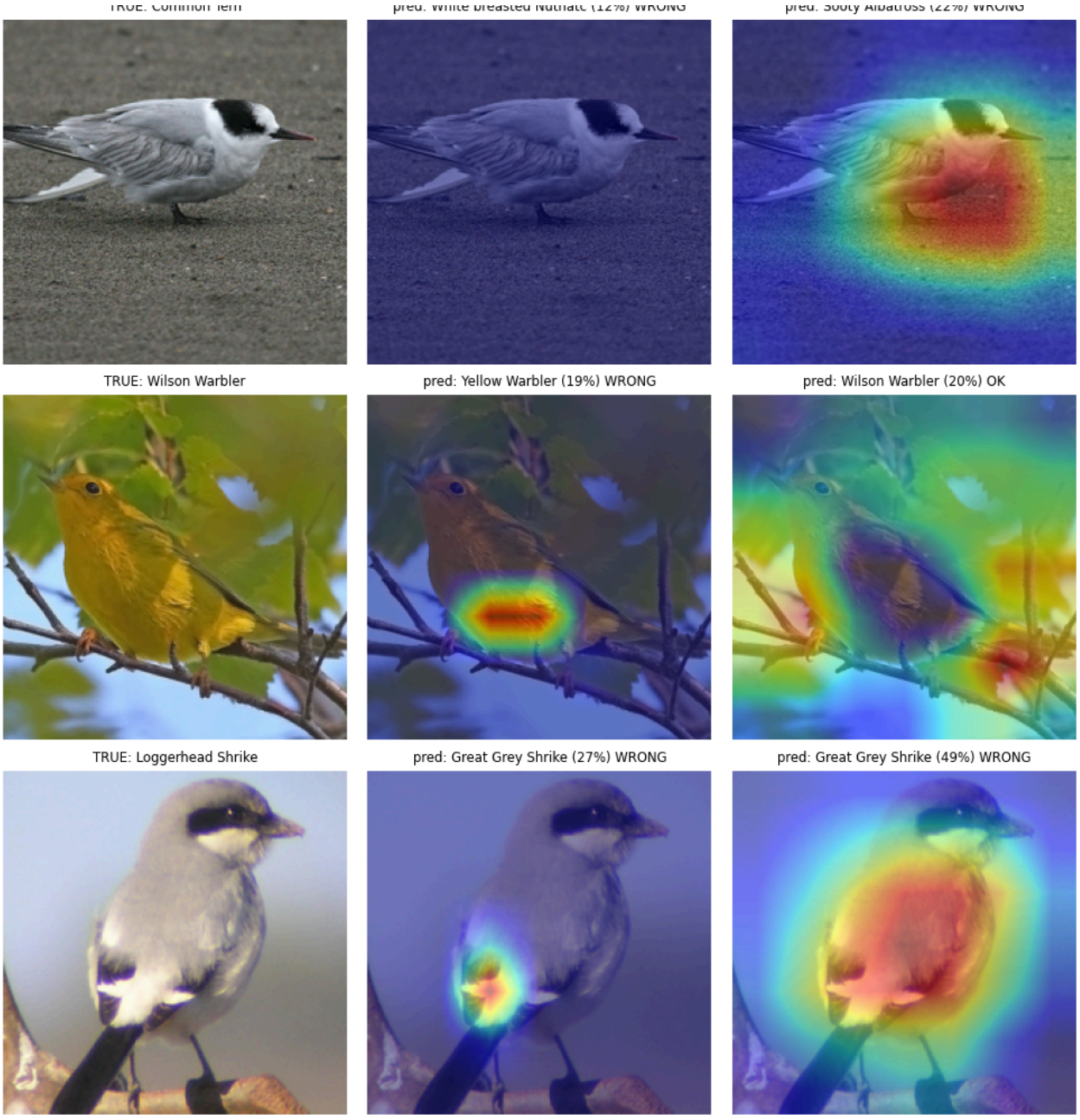

Grad-CAM on CORRECT predictions of cls-resnet50-ft (test set) - red = regions that drove the prediction
TRUE: Vermilion Flycatcher
[cls-resnet50-ft]
pred: Vermilion Flycatcher (60%) OK
[cls-scratch]
pred: Vermilion Flycatcher (80%) OK
TRUE: Red winged Blackbird
pred: Red winged Blackbird (89%) OK
pred: American Redstart (16%) WRONG
TRUE: Hooded Oriole
pred: Hooded Oriole (94%) OK
pred: Hooded Warbler (38%) WRONG



Test image	Fine-tuned ResNet-50	Scratch CNN
Vermilion Flycatcher	correct, 60%, red breast	correct, 80%, head and sky
Red-winged Blackbird	correct, 89%, red shoulder patch	wrong: American Redstart, 16%
Hooded Oriole	correct, 94%, head and body	wrong: Hooded Warbler, 38%
Pacific Loon	correct, 11%, almost flat map	correct, 22%, bird and water
Belted Kingfisher	correct, 91%, breast band and head	correct, 52%, head and sky
Horned Grebe	correct, 95%, head and neck	wrong: Eared Grebe, 28%

Figure: Grad-CAM on wrong predictions of the fine-tuned model (`gradcam_test_wrong.png`)





Test image	Fine-tuned ResNet-50	Scratch CNN
Caspian Tern	Common Tern, 21%, head and bill	Arctic Tern, 21%
Shiny Cowbird	Brewer's Blackbird, 47%, head	correct, 15%
Cape Glossy Starling	Boat-tailed Grackle, 49%, chest and legs	correct, 33%, bird and grass
Common Tern	White-breasted Nuthatch, 12%, flat map	Sooty Albatross, 22%, sand
Wilson's Warbler	Yellow Warbler, 19%, lower body	correct, 20%, leaves
Loggerhead Shrike	Great Grey Shrike, 27%, wing patch	Great Grey Shrike, 49%

What the maps show.

- **The fine-tuned model looks at the bird, often at the mark birdwatchers use.** Its maps are small and sit on the bird: the red shoulder patch of the Red-winged Blackbird, the breast band of the kingfisher, the head and neck of the grebe. The scratch CNN's maps are large blobs that cover the bird together with sky, water, grass or leaves.
- **The scratch CNN is sometimes right for the wrong reason.** For the Wilson's Warbler its map lies mostly on the leaves around the bird, and for the Cape Glossy Starling it includes the grass. This matches the cross-family confusions in section 8: without pretraining, the model leans on background and overall colour.
- **Its mistakes are understandable.** It calls the Red-winged Blackbird an American Redstart and the Hooded Oriole a Hooded Warbler: black birds with red or yellow patches. It sees the colours but not their exact pattern.
- **The fine-tuned model's mistakes are look-alikes, made with low confidence.** Its wrong predictions are glossy black birds (cowbird, blackbird, starling, grackle), two terns and two shrikes, all at 12–49% confidence; its correct ones are mostly at 60–95%. Its maps stay on the bird even when it is wrong, so the errors come from differences too fine for a 224-pixel image and a 7 × 7 map, not from looking at the wrong place. For the Caspian Tern it looks at the head and bill, where the large red bill gives the answer, but still picks Common Tern.
- **A flat map means low confidence.** For the Pacific Loon (correct, 11%) and the crouching Common Tern (wrong, 12%) the fine-tuned map is almost uniform: no region strongly supports the predicted class. The tern's black cap and pale grey body in this unusual pose do resemble a nuthatch.

These are 12 images chosen at random, so they illustrate the patterns but do not measure them. The overlap between the figure title and the column labels in the first version of the figures was fixed by switching the plot to constrained layout.

10. Benchmarking

On the test set our fine-tuned ResNet-50 reaches 83.1% at 224 px, on par with published fine-tuned ResNet-50 results at 448 px (82.5% and 84.5%). Our smaller scratch CNN reaches 50.4%, 1.5 to 5 points below published scratch ResNet-50 results.

Published results on the same dataset and the same official test split (top-1 test accuracy):

Source	Model	Training	Input	Accuracy
Barz and Denzler, WACV 2020, Table 2	ResNet-50	from scratch, cross-entropy	448 px	51.9 %

Source	Model	Training	Input	Accuracy
Barz and Denzler, WACV 2020, Table 2	ResNet-50	from scratch, cross-entropy + label smoothing	448 px	55.5 %
Barz and Denzler, WACV 2020, Table 2	ResNet-50	ImageNet, fine-tuned	448 px	82.5 %
Yang et al. (NTS-Net), ECCV 2018, Table 2	ResNet-50 baseline	ImageNet, fine-tuned	448 px	84.5 %

Our results on the same test split (top-1 test accuracy, 224 px):

Model	Training	Parameters	Accuracy	Closest published
Scratch CNN (ours)	from scratch, label smoothing	11.3 M	50.4%	55.5% (Barz and Denzler)
ResNet-50 linear probe (ours)	ImageNet, frozen	0.41 M trained	69.8%	no equal setup reported
ResNet-50 fine-tuned (ours)	ImageNet, fine-tuned	23.9 M	**83.1%**	82.5% (Barz and Denzler), 84.5% (NTS-Net baseline)

Our fine-tuned model is within the published range at a quarter of the input pixels. The standard error on 5,794 images is about 0.5 points, so 83.1% against 82.5% is a tie, not a win. Our scratch CNN is 5.1 points below the published scratch ResNet-50 with label smoothing, which uses the same loss trick, and 1.5 points below the plain one.

How the comparison was made. We compare directly against reported numbers. The dataset and the test split are the same (5,794 official test images), the metric is the same (top-1 accuracy), and, as in the papers, the test set was not used for any choice. The setups still differ in ways that explain the gaps:

1. **Resolution:** the papers use 448 × 448 inputs, we use 224 × 224, four times fewer pixels. Fine details such as eye rings and wing bars separate many species, so higher resolution usually helps.
2. **Training data:** we train on 5,094 images because 900 are held out for validation; the papers train on all 5,994.
3. **Model size:** our scratch CNN has 11.3 M parameters, about half of ResNet-50.
4. **Training length and recipe:** 50 epochs (scratch) and 30 epochs (transfer) with our own schedule; the papers use their own settings.

The gap between training from scratch and fine-tuning is about 30 points in the papers (51.9% → 82.5%) and 33 points in our test results (50.4% → 83.1%). Our experiments therefore reproduce the main published finding: on a small fine-grained dataset, ImageNet pretraining matters far more than the choice of training details.

11. Final model selection

We select the fine-tuned ResNet-50 (`outputs/runs/cls-resnet50-ft/best.pt` , epoch 26) as the Milestone 1 classifier and encoder.

- It is best on every validation metric: 81.7 % accuracy, 94.5 % top-5, 0.811 macro F1, 13.5 points above the linear probe and 35 points above the scratch CNN.
- The choice was made on the validation set only. The test set was evaluated once afterwards and confirms it: 83.1% accuracy, 95.6% top-5 and 0.830 macro F1, against 69.8% for the probe and 50.4% for the scratch CNN.
- It is ready for Milestone 2: its four stages (256, 512, 1,024 and 2,048 channels) can feed a U-Net decoder for colorization, and its 7 × 7 × 2,048 last stage can feed a caption decoder.
- The scratch CNN stays as the from-scratch baseline that the project requires, and as a second encoder to compare in Milestone 2.

12. Failures and limitations

Several first attempts did not work, and the models still overfit; we list what went wrong, why, and what we would try next.

What did not work at first, and how we fixed it.

Problem	Cause	Fix
First grayscale detector flagged 52 colour photos as grayscale	it measured average colour, and grey sky or water made colourful birds look grey	count the share of clearly coloured pixels; an image is grayscale only if fewer than 0.1 % have colour
First duplicate detector reported 51 pairs	a loose threshold (distance ⩽ 4) matched photos with similar plain backgrounds	confident duplicates = distance ⩽ 2, checked visually; 8 real pairs remain
Word splitting produced "a also", "as nd"	any split into two frequent words was accepted	split only if the two-word phrase exists in the captions

Problem	Cause	Fix
Some spelling fixes changed real words ("bicolored" → "colored")	the spell checker does not know bird vocabulary	a list of protected domain words
MLflow refused to log runs	MLflow 3 no longer accepts the folder store `./mlruns`	SQLite backend `mlflow.db`
Training on a MacBook was very slow (5 minutes before the first epoch)	the project folder is synced to iCloud, and data-loading workers re-read every file	trained on Google Colab T4 GPUs instead
Grad-CAM test on small synthetic images gave uniform maps	a 32-pixel input shrinks to a 1 × 1 last feature map	tested on 64-pixel inputs and against the exact formula

Limitations.

- **Overfitting:** training accuracy reaches 81 % (scratch), 94 % (probe) and 100 % (fine-tuned) against 47 %, 68 % and 82 % on validation. With about 25 images per species the models memorise the training set.
- **Resolution:** we train at 224 px; published fine-grained results use 448 px.
- **Training length:** each run's best epoch was at or near its last epoch, so longer training might add a little.
- **Data loading:** on Colab the CPU, not the GPU, limits speed, so experiments were slower than they needed to be.
- **Tuning:** to fit the time budget we tuned only the linear probe, for 5 epochs per trial, on three hyperparameters.

What we would try next: 448-pixel inputs for fine-tuning; stronger regularisation for the scratch CNN (mixup or CutMix, more dropout, longer training); tuning the fine-tuned model rather than the probe; caching resized images to remove the data-loading bottleneck; and part-based or attention models designed for fine-grained recognition.

13. Team contributions

Member	ID	GitHub	Contributions
Moaz Mohamed Zakaria	22P0307	Moazdarwishh	**Project setup:** repository structure, YAML configs, command-line interface and Colab notebook. **Data understanding and preparation (requirement 1.1):** train/validation split, exploratory data analysis, duplicate and grayscale checks, caption cleaning, data augmentation and data loaders. **Model development:** the CNN trained from scratch. **Hyperparameter tuning:** grid search. **Explainability:** Grad-CAM.
Ahmed Abbady	22P0308	AhmmedAbbady	**Model development and training (requirement 1.2):** the CNN trained from scratch, ResNet-50 transfer learning (linear probe and fine-tuning), the training loop, and evaluation with accuracy, top-5, macro precision, recall and F1. **Explainability:** Grad-CAM. **Report:** writing and analysis of the results.
Anas Mansour	22U0005	anasmansourr	**Model development and training (requirement 1.2):** the CNN trained from scratch, ResNet-50 transfer learning (linear probe and fine-tuning), the training loop, and evaluation with accuracy, top-5, macro precision, recall and F1. **Explainability:** Grad-CAM. **Report:** writing and analysis of the results.

Code: github.com/Moazdarwishh/DeepLearning-Project. Trained models and outputs: Google Drive.

14. References

1. Wah, C., Branson, S., Welinder, P., Perona, P., Belongie, S. The Caltech-UCSD Birds-200-2011 Dataset. Caltech, 2011.

2. Multimodal-Fatima/CUB_train and CUB_test, Hugging Face datasets.
3. He, K., Zhang, X., Ren, S., Sun, J. Deep Residual Learning for Image Recognition. CVPR 2016.
4. Selvaraju, R. R. et al. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV 2017.
5. Bergstra, J., Bengio, Y. Random Search for Hyper-Parameter Optimization. JMLR 13, 2012.
6. Szegedy, C. et al. Rethinking the Inception Architecture for Computer Vision (label smoothing). CVPR 2016.
7. Loshchilov, I., Hutter, F. Decoupled Weight Decay Regularization (AdamW). ICLR 2019.
8. Barz, B., Denzler, J. Deep Learning on Small Datasets without Pre-Training using Cosine Loss. WACV 2020.
9. Yang, Z. et al. Learning to Navigate for Fine-grained Classification (NTS-Net). ECCV 2018.
10. PyTorch and torchvision (ResNet-50 ImageNet weights), MLflow, Hugging Face Datasets, scikit-learn.